

Data Structures

Lecture 18: Search – Hashing

Soobin Um

May 18, 2026

Recap: Dictionary

Dictionary: 정의 및 구조

- 정의
 - 데이터를 Key-Value 쌍으로 저장하는 선형 구조
 - 배열 또는 연결 리스트는 위치(인덱스)를 통해 원소를 찾는 반면,
 - 딕셔너리는 key를 통해 원하는 값(원소)을 빠르게 찾을 수 있음
(즉, “key 기반의 리스트”와 같음)

Dictionary: 정의 및 구조

- 정의

- 데이터를 **Key-Value 쌍으로 저장**하는 선형 구조
 - 배열 또는 연결 리스트는 위치(인덱스)를 통해 원소를 찾는 반면,
 - 딕셔너리는 key를 통해 원하는 값(원소)을 빠르게 찾을 수 있음 (즉, “**key 기반의 리스트**”와 같음)

- 구조

- 내부적으로는 배열 구조. (**배열의**) 각 칸에 **key-value 쌍을 함께 저장**
- Value는 중복 가능하지만, **key는 유일해야 함**
- Key를 **해시함수(Hash function)**를 통해 (**배열의**) 인덱스로 변환
 - Key 자체를 배열의 직접적인 인덱스로 사용할 수는 없기 때문

Key → [Hash Function] → Index

Index	0	1	2	3	4	5	6	7
Slot	null	(2025001, "Alice")	(2025017, "Cindy")	null	(2023049, "Jack")	null	null	(2022057, "John")

주요 연산 및 특징

- 주요 연산

- 삽입 (put): 새로운 key-value 쌍 추가
- 탐색 (get): key를 통해 value를 찾음
- 삭제 (remove): 특정 key-value 쌍 제거
- 수정 (put): 동일 key에 새로운 value 할당
- 키 존재 여부 확인 (contains): 딕셔너리 상에서 특정 key의 존재 여부 확인

- 특징

- 빠른 탐색

- 일반적인 리스트에서 특정 값을 찾을 땐, 기본적으로 순차 탐색 필요 ($O(n)$)
 - 하지만 딕셔너리에선 (key를 이용하므로) 평균 $O(1)$ 에 탐색 가능

- 순서 없음

- 일반적인 리스트는 구조적으로 앞에서 뒤 또는 뒤에서 앞으로의 순서가 존재
 - 하지만 딕셔너리는 순서를 유지하지 않음 ($\rightarrow$ 정렬을 지원하지 않음)

- 해시 충돌 (hash collision)

- 서로 다른 두 개 이상의 key가 해시 함수를 통해 같은 인덱스로 변환되는 상황
 - 삽입/삭제/탐색 연산 모두 최대 $O(n)$ 의 비용이 발생할 수 있음

구현: HashMap 구조 이용

```
import java.util.HashMap;

public static void main(String[] args) {
    HashMap<String, String> dict = new HashMap<>();

    dict.put("2025001", "Alice");    // 삽입
    dict.put("2025017", "Cindy");
    dict.put("2023049", "Jack");
    dict.put("2022057", "John");

    System.out.println("2025017: " + dict.get("2025017"));    // 탐색

    dict.put("2025001", "Bob");    // 수정
    dict.remove("2023049");    // 삭제

    if (dict.containsKey("2025005"))    // 키 존재 여부 확인
        System.out.println("Exists")
}
```

시간 복잡도

	Search	Insert / Remove
Unsorted list (array)	$O(n)$	$O(1) / O(n)$
Sorted list (array)	$O(\log n)$	$O(n)$
Binary Search Tree	$O(\log n)$	$O(\log n)$
Hashing (or Hash Table)	$O(1)$	$O(1)$

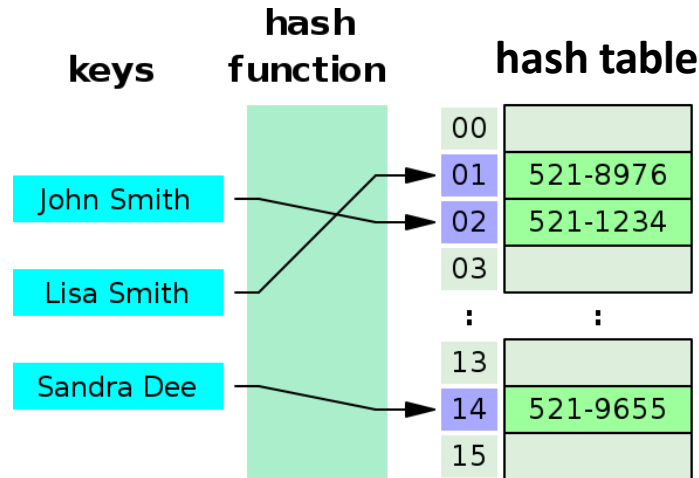
Hashing

Hashing (해싱)

- 해시 함수 $h(k)$ 를 이용하여, 키 k 를 해시 테이블의 특정 인덱스 또는 슬롯으로 매핑
 - 해시 테이블 HT : 아이템을 저장하는 크기 m 의 배열
 - 해시 함수 $h(\cdot)$: 키 k 를 $0 \sim (m-1)$ 범위의 값으로 매핑
 - Ex) $h(k) = k \% m$
- 키가 k 인 아이템은 해시 함수를 통해 얻은 해시 테이블의 인덱스 위치 $HT[h(k)]$ 에 저장됨

Hashing (해싱)

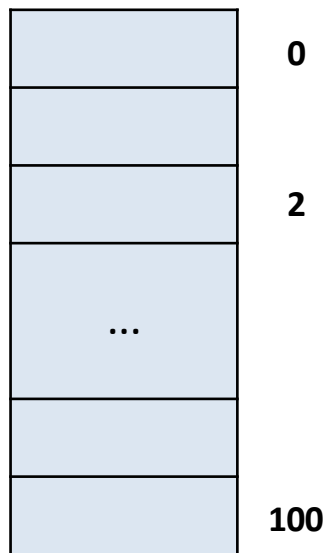
- 해시 함수 $h(k)$ 를 이용하여, 키 k 를 해시 테이블의 특정 인덱스 또는 슬롯으로 매핑
 - 해시 테이블 HT : 아이템을 저장하는 크기 m 의 배열
 - 해시 함수 $h(\cdot)$: 키 k 를 $0 \sim (m-1)$ 범위의 값으로 매핑
 - Ex) $h(k) = k \% m$
- 키가 k 인 아이템은 해시 함수를 통해 얻은 해시 테이블의 인덱스 위치 $HT[h(k)]$ 에 저장됨



https://en.wikipedia.org/wiki/Hash_table

해싱이 유용한 상황

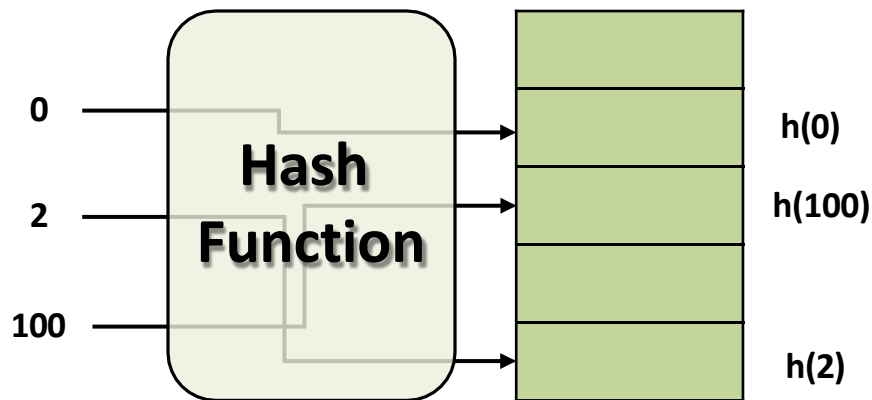
- 키 개수 n 은 작지만, 키의 범위 K_{max} 는 매우 큰 경우 (즉, $n \ll K_{max}$)
 - 단순 배열: $(K_{max} + 1)$ 크기의 배열 필요 (공간 복잡도: $O(K_{max})$) → 비효율적 (대부분의 공간 비게 됨)
 - 해싱: 작은 사이즈(e.g., m)의 해시 테이블만으로 저장/탐색 가능 (공간 복잡도: $O(m)$)
 - 보통 해시 테이블의 사이즈 m 은 키 개수 n 보다 조금 더 큰 정도임 ($1.3n \leq m \leq 2n$)



단순 배열

: 키의 범위에 해당되는
공간만큼의 배열 필요

vs.

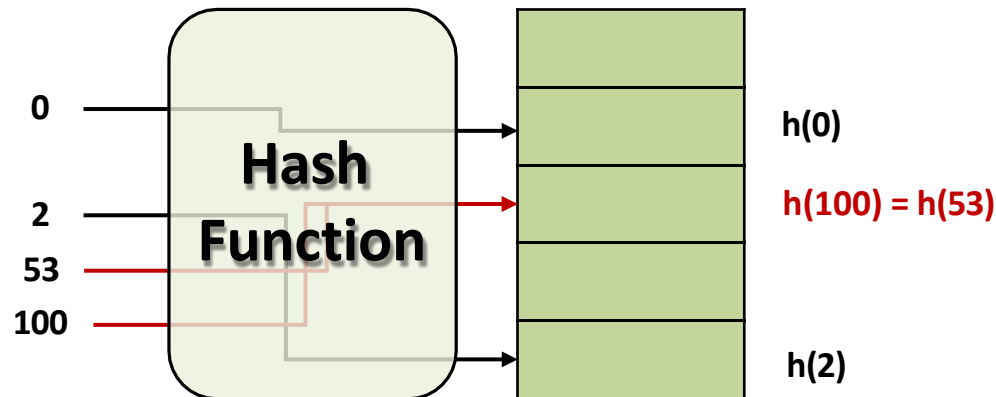


해싱

: 키 개수와 비슷한 공간의
배열이면 충분

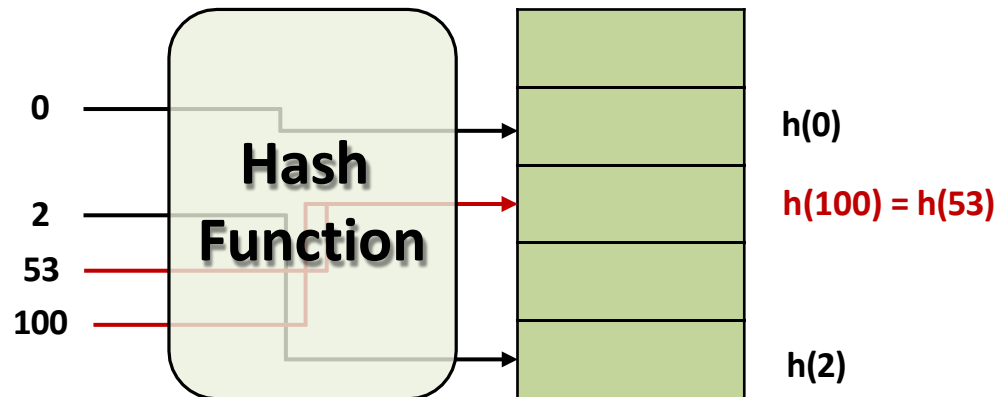
Collision (충돌)

- 두 개 이상의 서로 다른 키가 동일한 해시 함수 값을 갖는 상황
 - 동일한 슬롯에 여러 아이템들이 몰리면서 탐색, 삽입, 삭제 등의 연산에 추가적인 작업 필요하게 됨
 - 그 결과, 해시 테이블의 성능이 크게 저하될 수 있음
 - 충돌이 많아질수록 성능 저하 심화



Collision (충돌)

- 두 개 이상의 서로 다른 키가 동일한 해시 함수 값을 갖는 상황
 - 동일한 슬롯에 여러 아이템들이 몰리면서 탐색, 삽입, 삭제 등의 연산에 추가적인 작업 필요하게 됨
 - 그 결과, 해시 테이블의 **성능이 크게 저하**될 수 있음
 - 충돌이 많아질수록 성능 저하 심화
- 충돌을 줄이기 위한 기본 전략
 - **해시 함수를 잘 설계**함으로써, 충돌 발생 가능성을 최소화해야 함
 - 그럼에도, “**전체 키 개수 $n >$ 해시 테이블 크기 m** ”일 경우, **충돌은 반드시 발생**하게 됨 (비둘기 집 원리)
 - 따라서, 충돌 발생 시 이를 처리하기 위한 충돌 해결 방법(collision resolution)이 필수적



“좋은” 해시 함수의 조건

- 이상적으로 해시 함수는 “단순 균등 해싱 (Simple Uniform Hashing)” 가정을 만족해야 함
 - 모든 키는 m 개의 슬롯 중 어느 슬롯으로든 동일한 확률로 매핑되어야 함
 - 특정 슬롯에 치우치지 않도록 균등하게
 - 한 키가 어느 슬롯에 매핑되는지의 여부는, 다른 키들의 해싱 결과와 무관해야 함
 - 키들 간의 해싱 결과가 서로에게 영향을 주지 않아야

“좋은” 해시 함수의 조건

- 이상적으로 해시 함수는 “단순 균등 해싱 (Simple Uniform Hashing)” 가정을 만족해야 함
 - 모든 키는 m 개의 슬롯 중 어느 슬롯으로든 동일한 확률로 매핑되어야 함
 - 특정 슬롯에 치우치지 않도록 균등하게
 - 한 키가 어느 슬롯에 매핑되는지의 여부는, 다른 키들의 해싱 결과와 무관해야 함
 - 키들 간의 해싱 결과가 서로에게 영향을 주지 않아야
- 하지만 현실적으로, 키들이 어떤 분포와 함께 입력될 지 미리 알기 어려움
 - 이상적인 단순 균등 해싱 가정을 만족하는 해시 함수 설계가 쉽지 않음
 - 따라서 실제 시스템에선, 계산이 간단하면서 키들을 가능한 한 골고루 분포 시키는 함수를 선택

“좋은” 해시 함수의 조건

- 이상적으로 해시 함수는 “단순 균등 해싱 (Simple Uniform Hashing)” 가정을 만족해야 함
 - 모든 키는 m 개의 슬롯 중 어느 슬롯으로든 동일한 확률로 매핑되어야 함
 - 특정 슬롯에 치우치지 않도록 균등하게
 - 한 키가 어느 슬롯에 매핑되는지의 여부는, 다른 키들의 해싱 결과와 무관해야 함
 - 키들 간의 해싱 결과가 서로에게 영향을 주지 않아야
- 하지만 현실적으로, 키들이 어떤 분포와 함께 입력될 지 미리 알기 어려움
 - 이상적인 단순 균등 해싱 가정을 만족하는 해시 함수 설계가 쉽지 않음
 - 따라서 실제 시스템에선, 계산이 간단하면서 키들을 가능한 한 골고루 분포 시키는 함수를 선택

→ **Common choice:** Modular 연산 (e.g., $h(k) = k \% m$)

Modular Hash Function

- 해시 함수의 입력 즉, 키는 정수로 간주
 - 만약 키가 정수가 아닌 문자열 등의 형태일 경우, 정수로 변환하는 과정 수행

Modular Hash Function

- 해시 함수의 입력 즉, 키는 정수로 간주
 - 만약 키가 정수가 아닌 문자열 등의 형태일 경우, 정수로 변환하는 과정 수행
- ASCII 코드를 활용한 변환: 문자열 “CLRS”
 - ASCII 값: C = 67, L = 76, R = 82, S = 83
 - 28진법에 따라 정수로 변환: $(67 \cdot 128^3) + (76 \cdot 128^2) + (82 \cdot 128^1) + (83 \cdot 128^0) = 141,764,947$

Modular Hash Function

- 해시 함수의 입력 즉, 키는 정수로 간주
 - 만약 키가 정수가 아닌 문자열 등의 형태일 경우, 정수로 변환하는 과정 수행
- ASCII 코드를 활용한 변환: 문자열 “CLRS”
 - ASCII 값: C = 67, L = 76, R = 82, S = 83
 - 28진법에 따라 정수로 변환: $(67 \cdot 128^3) + (76 \cdot 128^2) + (82 \cdot 128^1) + (83 \cdot 128^0) = 141,764,947$
- $h(k) = k \bmod m$
 - 키 k 를 m 으로 나눈 나머지를 이용하여, 총 m 개의 슬롯 중 하나에 매핑 (가장 기본적인 방식)
 - 예: $m=12, k=100$ 일 경우, $h(100) = 100 \bmod 12 = 4$
 - 키 100은 해시 테이블의 슬롯 4에 저장

Modular Hash Function

- 해시 함수의 입력 즉, 키는 정수로 간주
 - 만약 키가 정수가 아닌 문자열 등의 형태일 경우, 정수로 변환하는 과정 수행
- ASCII 코드를 활용한 변환: 문자열 “CLRS”
 - ASCII 값: C = 67, L = 76, R = 82, S = 83
 - 28진법에 따라 정수로 변환: $(67 \cdot 128^3) + (76 \cdot 128^2) + (82 \cdot 128^1) + (83 \cdot 128^0) = 141,764,947$
- $h(k) = k \bmod m$
 - 키 k 를 m 으로 나눈 나머지를 이용하여, 총 m 개의 슬롯 중 하나에 매핑 (가장 기본적인 방식)
 - 예: $m=12$, $k=100$ 일 경우, $h(100) = 100 \bmod 12 = 4$
 - 키 100은 해시 테이블의 슬롯 4에 저장

Question: $h(k) = k \bmod 8$ 은 좋은 해시 함수인지?

$h(k) = k \bmod 8$: 좋은 해시 함수?

- **(문제점 1)** 키가 모두 짝수일 경우, hash table의 절반은 사용되지 못함
 - 키가 모두 짝수일 경우, $k \bmod 8$ 값이 홀수 슬롯에는 절대 등장하지 않게 됨
 - $k = 8 * i + j$ 에서 k 가 짝수일 경우, $k \bmod 8$ 에 해당되는 j 값은 반드시 짝수여야만 함
 - 해시 값 분포가 치우쳐져, 충돌 가능성 크게 증가

$h(k) = k \bmod 8$: 좋은 해시 함수?

- **(문제점 1)** 키가 모두 짝수일 경우, hash table의 절반은 사용되지 못함
 - 키가 모두 짝수일 경우, $k \bmod 8$ 값이 홀수 슬롯에는 절대 등장하지 않게 됨
 - $k = 8 * i + j$ 에서 k 가 짝수일 경우, $k \bmod 8$ 에 해당되는 j 값은 반드시 짝수여야만 함
 - 해시 값 분포가 치우쳐져, **충돌 가능성 크게 증가**
- **(해결책)** 해시 테이블의 사이즈 m 값을 소수(prime number)로 정함
 - 키들이 어떤 정수 c 의 배수일 때 (즉, $k = c * i$), m 과 c 가 서로소일 경우
 - 키들을 모두 서로 다른 슬롯에 할당시킬 수 있음.

서로소: 두 수의 공약수가 1밖에 없는 경우

$h(k) = k \bmod 8$: 좋은 해시 함수?

- **(문제점 1)** 키가 모두 짝수일 경우, hash table의 절반은 사용되지 못함
 - 키가 모두 짝수일 경우, $k \bmod 8$ 값이 홀수 슬롯에는 절대 등장하지 않게 됨
 - $k = 8 * i + j$ 에서 k 가 짝수일 경우, $k \bmod 8$ 에 해당되는 j 값은 반드시 짝수여야만 함
 - 해시 값 분포가 치우쳐져, 충돌 가능성 크게 증가
- **(해결책)** 해시 테이블의 사이즈 m 값을 소수(prime number)로 정함
 - 키들이 어떤 정수 c 의 배수일 때 (즉, $k = c * i$), m 과 c 가 서로소일 경우
 - 키들을 모두 서로 다른 슬롯에 할당시킬 수 있음.

서로소: 두 수의 공약수가 1밖에 없는 경우

※ 참고: 모순증명법을 통한 증명

$c \cdot i \equiv c \cdot j \pmod{m}$ 임을 가정 (모순 가정)

$\Rightarrow c \cdot i - c \cdot j \equiv 0 \pmod{m}$

$\Rightarrow c \cdot (i - j) \equiv 0 \pmod{m}$

이 때, c 와 m 은 서로소이므로 위의 식이 만족되기 위해선 $(i - j)$ 가 m 의 배수여야만 함 하지
만 i 와 j 는 $0 \sim (m-1)$ 범위의 서로 다른 수이므로, $(i - j)$ 가 m 의 배수일 수는 없음

따라서, 서로 다른 키 $c \cdot i$ 와 $c \cdot j$ 는 서로 같은 슬롯을 가리킬 수 없음 (즉, 위의 가정은 모순)

$h(k) = k \bmod 8$: 좋은 해시 함수?

- **(문제점 2)** 8과 같이 m 을 2의 거듭제곱이 사용될 경우, 해시 함수는 키 k 의 하위 3 비트만을 그대로 해시 값으로 사용하게 됨
 - $m = 2^p$ 일 경우, $h(k) = k \bmod 2^p$ 는 키의 하위 p 비트만을 사용하게 됨
 - 즉, 해시 함수가 키의 전체 비트를 고르게 활용하지 못하고, 하위 몇 비트에만 의존하게 됨
 - 키들의 하위 비트 패턴이 다양하지 않을 경우, 충돌 가능성 크게 증가
 - $k(19) = 10011$, $m = 1000 \Rightarrow h(k) = 011$
 - $k(27) = 11011$, $m = 1000 \Rightarrow h(k) = 011$
- **(해결책 1)** 해시 테이블의 사이즈 m 값을 소수(prime number)로 정함
 - 문제점 2에 대해서도 좋은 해결 방안

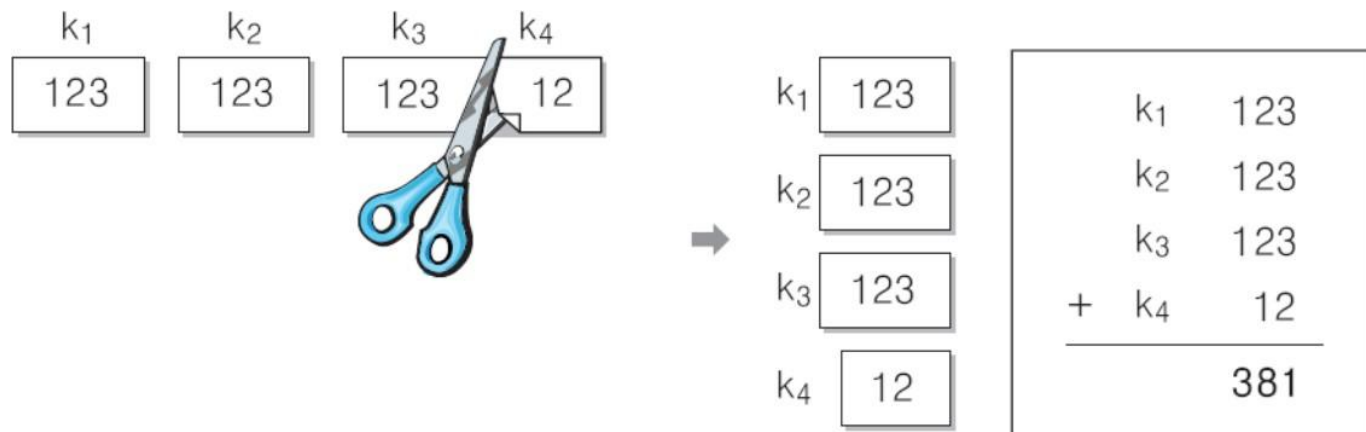
$h(k) = k \bmod 8$: 좋은 해시 함수?

- **(문제점 2)** 8과 같이 m 을 2의 거듭제곱이 사용될 경우, 해시 함수는 키 k 의 하위 3 비트만을 그대로 해시 값으로 사용하게 됨
 - $m = 2^p$ 일 경우, $h(k) = k \bmod 2^p$ 는 키의 하위 p 비트만을 사용하게 됨
 - 즉, 해시 함수가 키의 **전체 비트를 고르게 활용하지 못하고**, **하위 몇 비트에만 의존하게 됨**
 - 키들의 하위 비트 패턴이 다양하지 않을 경우, 충돌 가능성 크게 증가
 - $k(19) = 10011$, $m = 1000 \Rightarrow h(k) = 011$
 - $k(27) = 11011$, $m = 1000 \Rightarrow h(k) = 011$
- **(해결책 2) Mid-Square Method (중간 제곱법)**
 - 키를 제곱한 뒤, 가운데 r 개의 비트를 해시 테이블 사이즈 m 으로 나눈 나머지를 구함
 - 원래 키의 비트들이 잘 섞이게 되어, **입력 키의 패턴이 해시 함수 결과에 미치는 영향을 줄일 수 있음**

$$\begin{array}{r} 10100111 \\ X 10100111 \\ \hline 11011001110001 \end{array}$$

$h(k) = k \bmod 8$: 좋은 해시 함수?

- **(문제점 2)** 8과 같이 m 을 2의 거듭제곱이 사용될 경우, 해시 함수는 키 k 의 하위 3 비트만을 그대로 해시 값으로 사용하게 됨
 - $m = 2^p$ 일 경우, $h(k) = k \bmod 2^p$ 는 키의 하위 p 비트만을 사용하게 됨
 - 즉, 해시 함수가 키의 **전체 비트를 고르게 활용하지 못하고**, **하위 몇 비트에만 의존하게** 됨
 - 키들의 하위 비트 패턴이 다양하지 않을 경우, 충돌 가능성 크게 증가
 - $k(19) = 10011$, $m = 1000 \Rightarrow h(k) = 011$
 - $k(27) = 11011$, $m = 1000 \Rightarrow h(k) = 011$
- **(해결책 3) Folding Approach (접기 방법)**
 - 키를 t 개씩의 비트로 잘라서 합한 후, 해시 테이블 사이즈 m 으로 나눈 나머지를 구함
 - 키의 전체 정보가 고르게 섞이도록 하기 위함



Look ahead

Search – Hashing (2)